University of Waterloo

Faculty of Engineering

Department of Electrical & Computer Engineering

TerraScan

Group 2026.40

Arik Van Der Loo

Ernest Tang

James Byrne

Matthew Lee

Sougata Saha

William D. Bishop Consultant

July 12, 2026

Contents

1 HIGH LEVEL DESCRIPTION 3

1.1 Motivation 3
1.2 Project Objective 3
1.3 Block Diagram 4

2 PROJECT SPECIFICATIONS 6

2.1 Functional Specifications (Essential) 6
2.2 Functional Specifications (Non-Essential) 7
2.3 Non-Functional Specifications (Essential) 7
2.4 Non-Functional Specifications (Non-Essential) 8

3 DETAILED DESIGN 8

3.1 Sensor Module 8
3.2 FPGA 10
3.3 Microcontroller 20
3.4 Main PCB 22
3.5 Software System 25

4 DISCUSSION AND PROJECT TIMELINE 27

4.1 Evaluation of Final Design 27
4.2 Use of Advanced Knowledge 27
4.3 Creativity, Novelty, Elegance 28
4.4 Student Hours 28
4.5 Potential Safety Hazards 28
4.6 Project Timeline 29

REFERENCES 30

1 High Level Description

1.1 Motivation

Despite the growing prevalence of digital technologies in agriculture, accessibility remains a challenge for smaller farms. While eighty-one percent of large (five thousand acre & larger) operations use these tools, adoption drops to thirty-six percent for small farms under two thousand acres [1]. The biggest barrier to the use of such technologies on small farms has been the capital necessary to purchase them, and the fact that most existing technologies are developed primarily for large-scale industrial operation and thus do not provide small scale farmers personalized real-time information related to crop health [2].

Canopy density is typically one of the earliest and most reliable indicators that there may be stress on the crop, disease affecting the crop, or a reduced yield of an already stressed crop [3]. If canopy irregularities are not detected, they can accumulate over the growing season and can lead to potential economic losses. For small farms, the only alternative available for monitoring crops is to conduct manual scouting of all their crop fields [4]. Therefore, there exists a need for a portable, modular, and economical crop field scanning device that can provide farmers with automated metrics reflecting the health of their crop.

1.2 Project Objective

The objective of this project is to design an agricultural scanning device that provides real-time crop vegetation density and health metrics. The device is intended to address the lack of affordable precision agriculture tools available to small-scale farming operations. The device will use multiple sensing technologies, including Light Detection and Ranging (LiDAR) and near-infrared (NIR) spectroscopy, to collect spatial data and information on vegetation health. The device will include positional sensing in order to associate collected data with physical crop locations. To improve scanning range and speed, the device will be modular, allowing concurrent connection to several sensor modules. All data collected by the device will be organised, timestamped, and stored in a format that can be easily analysed by the customer. The device will be designed to be mounted on existing farming equipment and will take measurements as the equipment navigates the field.

1.3 Block Diagram

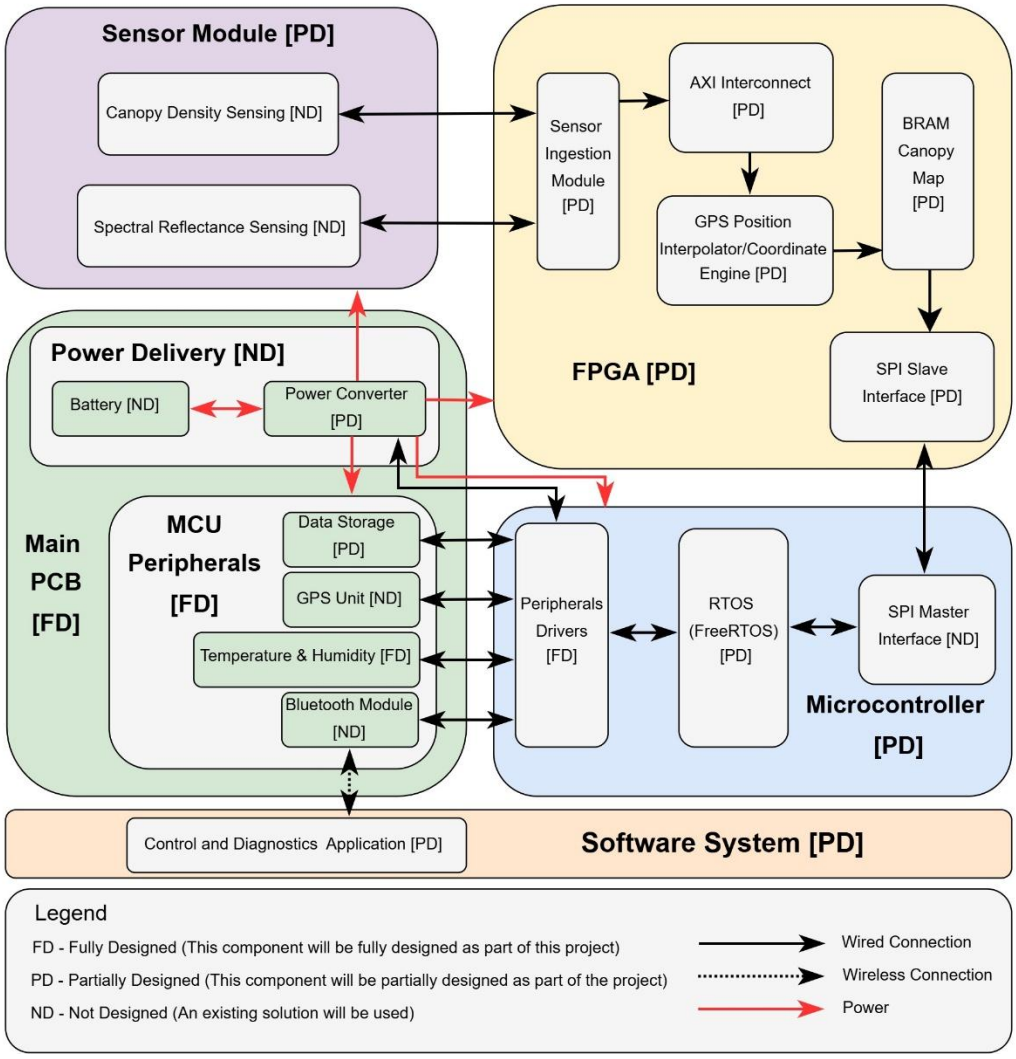


Fig. 1. High-level block diagram

1.3.1 Sensor Module

The sensor module subsystem consists of the physical hardware used to scan the crops. Table I gives a brief description of each component in the subsystem.

Table I
Sensor Module Components

Component	Description
Canopy Density Sensing	The primary optical distance sensing hardware positioned to scan vertical crop profiles.
Spectral Reflectance Sensing	The specialized light sensor used to measure reflected red and near infrared light from the plants.

1.3.2 FPGA (Field Programmable Field Array)

The FPGA subsystem handles the high-speed math tracking for the incoming sensor feeds. Table II gives a brief description of each component in this subsystem.

Table II
FPGA Components

Component	Description
Sensor Ingestion Module	Receives raw data streams from the LiDAR and spectral sensors and packages them into standardized packets, which are driven onto the internal AXI bus.
AXI Interconnect	The internal bus that carries packaged sensor data from the ingestion module to the processing engine, allowing multiple sensor streams to be arbitrated without a custom-built arbiter.
GPS Position Interpolator/Coordinate Engine	Estimates the sensor platform's position between GPS updates and converts each LiDAR polar reading into a 2D map coordinate relative to that position.
BRAM Canopy Map	An on-chip memory array that accumulates canopy density and NDVI values across a spatial grid representing the scanned area.
SPI Slave Interface	Sends processed canopy/NDVI data packets to the MCU and receives configuration and start/stop control signals from the MCU, over SPI.

1.3.3 Microcontroller

The microcontroller acts as the central coordinator for the device, handling timing and data movement. Table III gives a brief description of each component in this subsystem.

Table III
Microcontroller Components

Component	Description
MCU – FPGA SPI Interface	The MCU and FPGA communicate through an SPI interface. Data packets will be passed from the FPGA to the MCU, and control signals will be passed from the MCU to the FPGA.
Peripheral Drivers	Responsible for interfacing the microcontroller with external hardware components.
RTOS (FreeRTOS)	Acts as the central software orchestrator for the microcontroller. It manages system execution by dividing operations into prioritized tasks to ensure data collection and communication happen in real time.

1.3.4 Main PCB

The main PCB acts as the physical motherboard that holds the hardware peripherals and power hardware. Table IV gives a brief description of each component in this subsystem.

Table IV
Main PCB Components

Component	Description
Battery	Responsible for storing and providing power for all the electronic components in the system.
Power Converter	Steps down and/or regulates the raw battery voltage to the stable voltage levels required by the MCU, FPGA and peripheral hardware.
Data Storage	Saves and logs system metrics locally onto non-volatile storage, serving as the primary storage medium for all collected data.
GPS Unit	Provides real time geographical coordinates and timing data during device operations.
Temperature & Humidity	Measures ambient temperature and humidity of the scanning environment.
Bluetooth Module	Wirelessly connects scanning device to software system.

1.3.5 Software System

The software system allows the operator to interface with the unit remotely. Table V gives a brief description of each component in this subsystem.

Table V
Software System Components

Component	Description
Control and Diagnostics Application	A dashboard application running on an external mobile device that lets the user check device status and control the device.

2 Project Specifications

This section outlines the functional and non-functional specifications of the project. Both essential and non-essential specifications are listed, and all specifications are split into tables by category.

2.1 Functional Specifications (Essential)

Table VI lists the essential functional specifications ordered by subsystem.

Table VI
Essential Functional Specifications

Subsystem	Specification	Details
Sensor Module	**FS1:** LiDAR scanning pattern	The LiDAR sensor will scan at 10 Hz, returning distance points along a 2D plane for each scan. The module will be designed to be moving so that each 2D scan can be treated as a cross section of a 3D area.
Sensor Module	**FS2:** NIR sensor precision and accuracy	The wavelength spectrum data from the sensor must have sufficient accuracy and precision - it must capture 625-675 nm wavelengths for Red and 800-900 nm wavelengths for NIR to calculate a valid NDVI score [7].

FPGA	**FS3**: LiDAR ground detection	The detection of ground or plant is binary, and a distance reading below a threshold is counted as a plant hit. The sensor must be reliable up to 3m away.
FPGA	**FS4**: NDVI computation	The accelerator must compute NDVI and output an overall NDVI score for a scan. A density-weighted NDVI shall be returned once every metre the sensor moves.
Microcontroller	**FS5**: MCU to FPGA communication	The MCU must interface with the FPGA over SPI to read the data packets containing NDVI and Canopy density readings at a specified sampling rate.
MCU	**FS6**: Data storage	The MCU must be able to write all valid data packets to onboard non-volatile data storage without corruption.
MCU Peripherals	**FS7**: GPS accuracy	The global position sensing must be precise to at least 5 metres of horizontal accuracy when stationary.

2.2 Functional Specifications (Non-Essential)

Table VII lists the non-essential functional specifications ordered by subsystem.

Table VII
Non-Essential Functional Specifications

Subsystem	Specification	Details
MCU Peripherals	**FS8**: Temperature and Humidity accuracy	The sensor shall have an accuracy of at least ±1°C for temperature measurements and ±2% RH for humidity measurements.
MCU Peripherals	**FS9**: Device to Application Communication	The MCU will periodically send the current state of all configurable settings & state of the system to the application over Bluetooth. The application will communicate any setting changes and start/stop commands through Bluetooth as required.
Software Systems	**FS10**: Application	The application will display the current state of the system (scanning/not scanning) and the current settings. It will be able to be used to start/stop scanning and to set any user configurable settings.

2.3 Non-Functional Specifications (Essential)

Table VIII lists the essential non-functional specifications ordered by subsystem.

Table VIII
Essential Non-Functional Specifications

Subsystem	Specification	Details
Power Delivery	**NFS1:** Battery life	The battery must provide enough power to sustain at least 8 hours of field scanning on a single charge.
Sensor Module	**NFS2:** Sensor module decoupling	The NIR and LiDAR sensors must reside on a separate module that is distinct from the main MCU/FPGA board.
Entire Device	**NFS3:** Durability	The device must maintain structural and electrical integrity per the profiles defined in MIL-STD-810H

		Method 514.8 Annex C (Category 4) for vibration and Method 516.8 Procedure I for functional shock.
Entire Device	**NFS4:** Operational Temperature Range	The scanning device must operate reliably within an ambient temperature range of 0 to 45 degrees Celsius.
Data Storage	**NFS5:** Removable data storage media	The system must log data onto a compact, non-volatile storage media formatted to a standard file system. The storage media will be removable to easily transfer files.

2.4 Non-Functional Specifications (Non-Essential)

Table IX lists the non-essential non-functional specifications ordered by subsystem.

Table IX
Non-Essential Non-Functional Specifications

Subsystem	Specification	Details
Power Delivery	**NFS6:** Power Consumption	The average power consumption of the device will be less than 20 W.
Entire Device	**NFS7:** Device Size	The control module will fit within a 20 x 20 x 20 cm area. The sensor module will fit within a 15 x 15 x 15 cm area.
Enclosure	**NFS8:** Ingress Protection	The enclosed device will have an ingress protection rating of IP65.
Sensor Module	**NFS9:** Sensor multi node architecture	The architecture must allow multiple sensor modules to operate simultaneously.

3 Detailed Design

3.1 Sensor Module

The Sensor Module consists of 3 components: The Canopy Density Sensor (RPLIDAR C1), Spectral reflectance sensor (AS7341), and the Sensor interface for both the sensors to communicate with the FPGA. RPILIDAR C1 communicates over TTL UART and AS7341 communicates over I2C. For the sensors, each part below was chosen over at least one alternative that was seriously considered and ruled out. These reasonings are summarized in the section below before more details are shown about the selected parts.

3.1.1 Part Decision

Canopy Density Sensor: The RPLIDAR A1M8-R6 was the original choice mainly because of the sample frequency and scan rate of the sensor. This option was discarded because after looking into it in more detail, it was found that it can't work efficiently with direct exposure to sunlight. The chosen sensor is RPLIDAR C1 which uses Direct Time of Flight (DTOF) Laser ranging which is much better than the first option because it can efficiently isolate a single, powerful laser pulse from all the solar noise [19]. The second reason why this option was selected was because of the compact size of the sensor, which helps us meet the (**NFS7**).

Spectral Reflectance Sensor: The original choice was AS7263 because it has 6 channels to measure different wavelengths, including a dedicated channel for Near Infrared (NIR), which is

needed for NDVI computation. It was discarded, because it didn't have a channel that measures the wavelength of Red Light, which is also required for NDVI computation, therefore not satisfying (**FS2**). The selected sensor is AS7341 because it has 11 channels and has dedicated channels to measure the wavelength of both NIR and Red Light, therefore satisfying (**FS2**).

3.1.2 Sensor Interface

The sensor interface on the sensor module side is the physical communication pathway that carries raw data streams from the RPILIDAR C1 and AS7341 sensors out of the sensor module to the FPGA, along with all power and ground lines needed to operate both sensors.

3.1.2.1 RPILIDAR C1 Physical Connector

As specified in the RPLIDAR C1 datasheet [11], the sensor ships with an XH2.54-5P male connector. Only 4 of the 5 pins are used. The pin assignment from the datasheet is as follows:

Table X
RPILIDAR Connections

Pin	Wire Colour	Signal	Type	Voltage
1	Red	VCC	Input Power	4.8-5.0 V
2	Yellow	TX	Output	0-3.5 V
3	Green	RX	Input	0-3.5 V
4	Black	GND	Ground	0 V

So, the sensor interface must have a XH2.54-5P female connector. The 5V VCC line must come directly from the power converter. The datasheet [11] specified a typical operating current of 230mA and a start-up current of up to 800mA, both of which exceed what any logical board regulator pin can supply. It also mentions a maximum power supply voltage ripple of 150mV, therefore the 5V rail must be clean and well-regulated to avoid measurement inaccuracies.
The TX and RX signal lines operate at 3.3V TTL as per the datasheet [11] and connect directly to the FPGA GPIO pins without any level shifting required.

3.1.2.2 AS7341 Physical Connector

The AS7341 comes in a breakout board (Adafruit 4698) with standard through-hole header pins. The pin assignments are as follows:

Table XI
AS7341 Connections

Pin	Type	Voltage
VIN	Input Power	3.0-5.0 V
GND	Ground	0 V
SCL	I2C Clock	3.0-5.0 V
SDA	I2C Data	3.0-5.0 V
INT	Interrupt to send signals	-
GPIO	GPIO	-

The Adafruit 4698 breakout board includes an onboard voltage regulator that accepts 3.0-5.0 V on VIN and steps it down to the 1.8 V required by the AS7341 chip internally, as documented in the Adafruit product guide [20]. In this design, 3.3V is supplied to VIN from the power converter. The

breakout board also includes onboard logic level shifting on SCL and SDA, making the I2C lines compatible with 3.0-5.0 V. This means the FPGA's 3.3V GPIO signals connect directly to SCL and SDA without any additional level shifting required. Connection to the interface is made via standard female-to-female jumper wires.

3.1.2.3 Communication Speed and Cable Length Constraints

The UART interface between the RPLIDAR C1 and the FPGA operates at 460800 baud as specified in the datasheet [11], meaning each bit occupies approximately 2.17 µs (1/460800). The UART lines don't carry any clock signal, however corrupted frames can be detected and discarded using the checksum byte included in every frame, preventing invalid readings from entering the Canopy Density Algorithm. The interface cable should still be kept as short as possible to minimize noise pickup and signal degradation, which will be determined after experimenting with the parts.

The I2C interface for AS7341 operates in Fast Mode at 400 kHz which requires a maximum clock and data rise time of 300 ns [12]. Rise time is determined by the pull-up resistance and the total bus capacitance: T = 0.8473 X RC [21]. With the onboard 10K ohm pull-up resistors on the Adafruit 4698 breakout board [20], the maximum allowable bus capacitance before violating the 300 ns limit is approximately 35 pF. There is no readily available information about the capacitance of jumper wires as they differ based on the length and the manufacturer, but it must be measured before using to make sure it's not more than 35 pF.

3.2 FPGA

The FPGA subsystem consists of a sensor ingestion module, a processing engine, an internal AXI bus, and an interface to the microcontroller subsystem. Data flows from the sensor subsystem, is packaged by the ingestion module, sent over the local interconnect, processed and mapped out by the engine, and a packet is sent to the microcontroller subsystem every one metre the sensor moves.

The subsystem shall be run on an AMD Artix 7 XC7A35T FPGA. The Artix-7 contains 215K logic cells, 13 Mb of BRAM, and 740 DSP slices [9]. The Artix-7 family is designed for low-power applications that require serial transceivers and high logic throughput [9], making this FPGA suitable for our plant health metric computation and module interfacing needs.

3.2.1 Sensor Ingestion Module

The main purpose of the sensor ingestion module is to receive and package the raw data from the RPLIDAR C1 and AS7341 into a standard data format that is used internally by the FPGA subsystem and passed onto an AXI bus as an AXI master.

The module consists of 3 independent sub-components: the LiDAR UART Parser, the NIR I2C Parser, and the GPS SPI Receiver. The LiDAR UART Parser and the NIR I2C Parser each act as an AXI mater to produce output packets that are driven onto the AXI bus. The GPS SPI Receiver does not produce AXI packets directly, instead it maintains internal registers to store the most recently received GPS data, which both parsers read from when assembling their output packets.

A shared 32-bit counter driven by the FPGA's 12MHz clock acts as the time reference across all 3 sub-components. Every packet produced by the module carries a timestamp from this counter, recording the FPGA clock cycle at which that sensor reading was captured. This is essential for the GPS Position Interpolator, which must correlate LiDAR data points with the GPS updates.

3.2.1.1 LiDAR UART Parser

The LiDAR UART Parser interfaces with the RPLIDAR C1 over TTL UART at 460800 baud, 8N1 framing. The parser consists of 2 layers: a bit-level UART receiver state machine and a packet framing layer.

The RPLIDAR C1 operates on a request/response model and does not transmit scan data autonomously after power-up. On system initialisation, the parser follows the startup sequence defined in the protocol document [16]. First, a GET_HEALTH request is transmitted to confirm the sensor is operational and not in Protection Stop state. If the sensor is in Protection Stop state, a RESET request is transmitted, and the parser waits at least 500ms before retrying. Once health is confirmed, a SCAN request is transmitted to initiate continuous scan data output. The parser then waits for the 7-byte response descriptor confirming the sensor has entered the scanning state, after which output data begins streaming continuously once the motor rotation stabilises.

The first 2 bytes of the response descriptor are fixed sync bytes identifying a valid response. Byte 5 encodes a Send Mode of 0x1 confirming Single Request/Multiple Response mode, meaning the LiDAR will continuously stream 5-byte measure packets without requiring further requests. Byte 6 is the SCAN data type identifier confirming incoming packets are scan measurement data [16].

The packet framing layer groups incoming bytes into 5-byte data packets per the RPLIDAR C1 communication protocol [16]. Each packet is validated using 2 integrity checks: C must always be 1, and S' must always be the bitwise inverse of S. Packets failing either check are discarded. Also, a distance_q2 value of 0 indicates an invalid measurement and is discarded.

For each valid packet, fixed-point fields are converted to physical values as follows [16]:
Actual Angle (degrees) = angle_q6 / 64.0
Actual Distance (mm) = distance_q2 /4.0

The parser then extracts all the measurement data, and the START flag from each valid data response packet. The parser then reads the current GPS data from the GPS SPI Receiver internal registers and assembles output packet for the AXI bus as follow:

Table XII
Output Packet Structure

Field	Width (bits)	Description
Distance (mm)	16	Distance measured in mm
Angle (degrees)	15	Angle measure in degrees
Start Flag	1	Star flag indicating a new measurement
velN	32	North velocity value of the GPS received from MCU in mm/s
velE	32	East velocity value of the GPS received from MCU in mm/s
headMot	32	Heading value of the GPS received from the MCU in degrees
Hardware Timestamp	32	FPGA clock counter value at time of packet receipt
Total	160	

The parser asserts TVALID for one clock cycle per valid packet, asserts TLAST simultaneously, and monitors TREADY for the backpressure from the data processing engine.

3.2.1.2 NIR I2C Parser

The NIR I2C Parser interfaces with the AS7341 sensor (Adafruit 4698 breakout) over I2C at 400kHz Fast Mode. The I2C address of the AS7341 is 0x39 [12]. The parser consists of 2 layers: an I2C master controller and a channel extraction layer.

The AS7341 enters a SLEEP state after power-up and does not perform any measurement autonomously. On system initialisation, the parser performs a configuration sequence by writing to the specific AS7341 registers which are mentioned in the datasheet over I2C [12].

The AS7341 has 11 spectral channels, but only 6 internal ADCs (CH0-CH5) can be used. The SMUX maps 6 channels to 6 ADCs at a time. The SMUX must be configured before any spectral measurement is started. The SMUX configuration is done in 2 cycles, the first cycle is for channels F1-F4, NIR and Clear and the second cycle is for channels F5-F8, NIR and clear. For NDVI computation, we need the channels F7 and NIR. The channel F7 measures wavelength centered at 630nm, which meet the (**FS2**) requirement. The NIR channel is centered at 910nm, which falls just outside the range 800-900nm specified in (**FS2**), but this is a hardware constraint. The 910nm NIR channel is the closest available on the AS7341 and is still expected to perform efficiently for our purpose. Since we need F7 and NIR, the SMUX is configured in the second cycle. The SMUX configuration is implemented as per the open-source Adafruit AS7341 library [17], and F7 is configured to CH2 and NIR is configured to CH5.

Once in the ACTIVE state, the parser polls the AVALID bit (bit 6) of the STATUS 2 register at address 0xA3. When AVALID = 1, the measurement cycle is complete, and channel data is ready to be read.

The channel data extraction layer reads the F7 and NIR channel data registers in bursts after detecting AVALID = 1. The low byte of each channel must always be read first as it triggers a 16-bit latch, followed immediately by reading the high byte. The specific registers and addresses for the required channels can be found from the datasheet [12].

The parser extracts F7 and NIR raw 16-bit ADC values from each completed measurement cycle. Then, the parser reads the current GPS data from the GPS SPI Receiver internal registers and assembles an output packet for the AXI bus as follows:

Table XIII
Output Packet Structure

Field	Width (bits)	Description
F7 (Red) Channel	16	Raw ADC value from CH2
NIR Channel	16	Raw ADC value from CH5
velN	32	North velocity value of the GPS received from MCU in mm/s
velE	32	East velocity value of the GPS received from MCU in mm/s
headMot	32	Heading value of the GPS received from the MCU in degrees
Hardware Timestamp	32	FPGA clock counter value at time of measurement completion
Total	160	

The parser asserts TVALID for one clock cycle per valid packet, asserts TLAST simultaneously, and monitors TREADY for the backpressure from the data processing engine.

3.2.1.3 GPS SPI Receiver

The SPI slave receiver accepts the incoming GPS packets from the MCU and immediately appends a 32-bit hardware timestamp when received. The receiver also updates its internal registers with the incoming GPS data. The registers are initialised to 0 at power-up, and then updated as the data comes in.

Table XIV
SPI Slave Receiver Register Map

Field	Width (bits)	Description
lon	32	Longitude position received from MCU in degrees
lat	32	Latitude position received from MCU in degrees
velN	32	North velocity value received from MCU in mm/s
velE	32	East velocity value received from MCU in mm/s
headMot	32	Heading value received from MCU in degrees
Hardware Timestamp	32	FPGA clock counter value at the time of SPI packet receipt
Total	192	

3.2.2 Data Processing Engine

After receiving a data packet from the sensor ingestion module, the data processing engine must update the BRAM canopy map with the received LiDAR and NIR. This section explains how the data flows through the different stages of the processing engine. The output packet is sent to the MCU every 1000mm (1m).

3.2.2.1 NDVI Compute Algorithm

To calculate the NDVI score for a given sensor reading, we require the Red and NIR channel data from the data packet received from the sensor ingestion module via the AXI bus. The following is the equation we implement in the processing engine to obtain the NDVI from a given sensor packet [5], [6]:

$$NDVI = \frac{NIR - Red}{NIR + Red} \tag{1}$$

NIR and Red are received from the AS7341 as two data bytes as described in Section 3.2.1.2, which we use to compute NDVI. We represent NDVI as a 16-bit fixed point value, which is sent as a 16-bit word in the summary packet that the FPGA subsystem outputs.

The NIR sensor is fixed on the device and does not sweep horizontally like the RPLIDAR C1 sensor, so we only receive NIR data on material directly under the sensor. This approach suffices as the sensor will very often be operated directly above the plant of interest, and the loss of using a single sensor over multiple should not be significant. An alternative we considered is using an array of sensors, which would provide full NIR coverage of the area the LiDAR sensor covers. The increase in cost and complexity to purchase multiple sensors and to map the NIR readings from multiple AS7341 sensors is not justifiable when one sensor is sufficient.

The half cone angle of the AS7341 sensor is 40 degrees and the photodiode array is square [12], so we can determine the ground area coverage using a simple trigonometric calculation. For sensor height h, the length and width of the NIR sensor coverage is:

$$l_{NIR} = h * \sin(80) \tag{2}$$

We may assume this coverage is roughly square then accumulate the NDVI readings for all BRAM array elements within this area. The mapping of the sensor readings is further elaborated in Section 3.2.2.4.

Each sensor packet corresponds to a NDVI reading, and each element in the BRAM map that receives more than one NDVI reading will have a NDVI score that accounts for multiple sensor readings. For a given BRAM array element (X, Y), we store an aggregate NDVI with an Infinite Impulse Response Filter using an Exponential Moving Average (EMA) formula:

$$NDVI_{new} = \alpha * NDVI_{sample} + (1 - \alpha) * NDVI_{old} \quad (3)$$

$NDVI_{new}$ is the new aggregated NDVI for the array element, $NDVI_{old}$ is the previous aggregated NDVI for the array element, $NDVI_{sample}$ is the NDVI reading corresponding to the current NIR sensor packet, and α is the smoothing factor. We choose α=0.25 to allow the use of shifters to implement the division by 4, but this parameter is adjustable by powers of 2 to improve accuracy and reduce noise based on the expected movement speed of the sensor.

3.2.2.2 GPS Position Interpolator

The GPS Position Interpolator maintains an estimate of the sensor platform's position on a relative 2D coordinate grid, where the position of the first received GPS packet defines the origin (0,0). All subsequent positions are expressed as cumulative displacements in millimeters from the origin. Without position interpolation, all LiDAR data points arriving between GPS updates would be assigned to the same outdates coordinate, producing incorrect canopy map.

The sensor platform is assumed to moves in a straight line along the Y-axis which is the direction of travel through the field. The platform velocity magnitude is computed from the North and East velocity components as follow:

$$vel\,(mm\,/\,s) = \sqrt{velN^2 + velE^2} \quad (4)$$

Since the Artix-7 does not support native floating-point arithmetic [10], this square root is calculated using the Xilinx CORDIC IP block configured in Square Root mode, consistent with fixed-point Q16.16 format which is used throughout the pipeline [13].
The elapsed time between consecutive LiDAR data points is computed using the hardware timestamps carried in each LiDAR AXI bus packet.

$$\Delta\, ticks = timestamp_{current} - timestamp_{previous} \quad (5)$$

At 12MHz each clock tick is 1/12,000,000 seconds. The cumulative Y displacement is then updated on every incoming LiDAR packet as follows:

$$Disp_{new} = Disp_{old} + (vel\, X\, \Delta\, ticks)\,/\,12{,}000{,}000 \quad (6)$$

All the arithmetic operations are implemented using the Xilinx CORDIC IP block. These IP blocks are verified, optimised for the Artix-7 architecture, and integrate directly with the AXI bus via their native AXI4-Stream interfaces [13].

When Disp reaches 1000mm, the interpolator asserts END flag HIGH for one clock cycle. This signals the BRAM Canopy Map to complete its canopy density and NDVI computations for the current 1 metre strip and forward the result to the MCU. Disp is then reset to 0, to begin measurement for the next 1 metre grid.

The headMot field from the embedded GPS data is monitored on every incoming LiDAR packet. The heading of the first received GPS packet is stored as the reference heading. On every subsequent packet, the absolute deviation from the reference heading is computed as follow:

$$\Delta\, heading \;=\; \left|headMot_{current} \;-\; headMot_{ref}\right| \tag{7}$$

If the heading deviation exceeds a threshold for a certain consecutive number of data points (which will be determined empirically), the platform is considered to be turning at the end of a row. When a turn is detected, all incoming LiDAR packets are discarded until the heading returns within threshold, Disp is reset to 0 establishing a new origin for the next row, a FLUSH signal is set HIGH for one clock cycle, signalling the BRAM canopy map to clear all stored data, and the reference heading is updated to the new heading once the turn complete and the straight line motion resumes.

Q16.16 is chosen over smaller formats such as Q8.8 because the GPS interpolator computation requires multiplying vel by the elapsed time. For this multiplication to produce correct result, the fixed-point format must be able to represent the full range of the velocity values without overflow. Q8.8 provides only 8-bit integer part, so the max value that can be represented $2^{16} – 1 = 255$ mm/s (0.9 km/h) which is very slow if we think about mounting the device on a tractor that moves in much higher speeds. Whereas in Q16.16, the max value that can be represented is $2^{32} –1 = 65535$ mm/s (236 km/h) which represents much closer values to the speed of a moving tractor.

Larger formats like Q32.32 is not used despite offering greater precision is because it requires a 32-bit integer and fractional part, totalling to 64 bit. This exceeds the 48-bit accumulator width of the DSP48E1 slice, which is a dedicated hardwired multiply-accumulate block built into every Artix-7 FPGA [18]. So, the complete arithmetic operations can't be completed in a single clock cycle with larger Q formats.

The GPS Position interpolator data packet for the AXI bus is structured as follow:

Table XV
AXI Bus Interpolator Data Packet

Field	Width (bits)	Description
Distance (mm)	16	Distance measured in mm
Angle (degrees)	15	Angle measure in degrees
Start Flag	1	Star flag indicating a new measurement
Disp	32	Relative position of the sensor platform in a 2D coordinate
headMot	32	Heading value of the GPS received from the MCU in degrees
FLUSH	1	Set HIGH for one clock signal when a turn is detected
END	1	Set HIGH for one clock signal when Disp reaches 1000mm
Hardware Timestamp	32	FPGA clock counter value at time of the LiDAR data point
Total	130	

3.2.2.3 CORDIC Coordinate Engine

The CORDIC coordinate engine calculates the position of each LiDAR point relative to the position of the sensor platform. Since we receive the angle of the LiDAR sensor for each data point, we can conduct a trigonometric calculation using a CORDIC engine to get the position of the LiDAR data.

To achieve this calculation, we use a Xilinx CORDIC IP to convert the raw polar coordinates from the sensor to get rectangular coordinates. We decided to use an IP that is verified and optimized by Xilinx instead of a manual implementation, because trigonometric operations are usually

hardware-intensive, complex, and require approximations that are difficult to implement and verify. By using an AXI4-Stream compliant CORDIC IP block, we can easily integrate this computation into our subsystem for coordinate calculation.

We wish to perform the following computations:

$$X_{data} = r * \cos(\theta) \quad (8)$$

$$Z_{data} = r * \sin(\theta) \quad (9)$$

Where X_{data} is the horizontal position on the canopy map, r is the distance reading from the LiDAR sensor, θ is the angle reading from the LiDAR sensor, and Z_{data} is the vertical height component of the LiDAR distance reading. We don't perform calculations for Y_{data}, since the sensor is fixed and the forward-backward position is captured by the GPS position interpolator.

The CORDIC engine requires fixed point twos-complement numbers for the Cartesian inputs and scaled radians (-1.0 to 1.0 representing -180° to 180°) for the angle inputs, and it must be configured to the Sin and Cos functional mode. We retain the same numerical representation as the GPS position interpolator (Q16.16) for the Cartesian inputs, and we must convert the angle data of the LiDAR sensor from degrees to scaled radians.

The following table describes the signals for the CORDIC IP core (excluding the global clock, resetn, and enable signals) [13]:

Table XVI
CORDIC Coordinate Engine Pinout and Signal Map

Name	Direction	Description
s_axis_{cartesian, phase}_tvalid	Input	Valid signal for input
s_axis_{cartesian, phase}_tready	Output	Ready signal for input
s_axis_{cartesian, phase}_tdata	Input	Cartesian/phase data input
m_axis_dout_tvalid	Output	Valid signal for output
m_axis_dout_tready	Input	Ready signal for output
m_axis_dout_tdata	Output	(Cos(θ), Sin(θ)) output vector

Our engine implementation will also include an adder to calculate X_{data}, since the CORDIC IP block only calculates the trigonometric operation to get the horizontal offset of the data point relative to the sensor position.

We maintain two global counters tracking the number of plant hits and the total number of LiDAR data points, represented as 32-bit unsigned integers. These counters shall be sent to the MCU in the output packet, as described in Section 3.2.4, to calculate the overall plant density of the scanned area. Every LiDAR reading increments the total number of LiDAR data points, but a plant hit is determined by if Z_{data} is below a configurable height threshold $h_{threshold}$, meaning the data point is above the height we consider ground and is classified as plant canopy.

3.2.2.4 BRAM Canopy Map

Using the outputs of the CORDIC coordinate engine, X_{data} and Y_{data}, we get the location of each LiDAR and NIR data point. To map and track the data at each physical location to a virtual location, we maintain a BRAM map acting as a spatial accumulator. Each element of the 2D array is uniform and represents an area of the land being scanned.

To determine the resolution of the canopy map, we mainly consider the memory footprint of the array. We decided that each output packet should represent a scan that covers a 1 metre by 1 metre

area. The grid size and resolution should be powers of 2 to allow the use of shifters instead of complex multipliers and dividers for map addressing and coordinate quantization respectively.

Each grid element shall be 1.6 cm by 1.6 cm, and the grid will be 64 by 64 elements to cover an area of 1.024 metres by 1.024 metres. This uses 4096 cells, and each cell shall be 32 bits to store a 16-bit NDVI and a 16-bit number of plant hits. This memory layout consumes $4096 * 32\ bits * \frac{1\ Kilobit}{1000\ bits} = 131.1Kb$, and the Artix-7 35T has 13 Mb of BRAM [9]. Using this layout allows us to have a meaningful data resolution while not consuming an excessive amount of BRAM cells. Using a lower resolution (3.2 cm by 3.2 cm grid and 1024 elements consuming $1024 * 32\ bits * \frac{1\ Kilobit}{1000\ bits} = 32.8Kb$) saves memory but may provide less meaningful data, and using a higher resolution (0.8 cm by 0.8 cm grid and 16384 elements consuming $16384 * 32\ bits * \frac{1\ Kilobit}{1000\ bits} = 524.3Kb$) may not yield much better data to be worth the memory overhead.

We must perform quantization on the X_{data} and Y_{data} coordinates calculated by the CORDIC coordinate engine and GPS position interpolator to determine the grid element each data point belongs to, and the quantization to get the grid coordinate is:

$$X_{grid} = round\left(\frac{X_{data}}{2^{16}} * 64\right) \tag{10}$$

$$Y_{grid} = round\left(\frac{Y_{data}}{2^{16}} * 64\right) \tag{11}$$

For NIR readings, we use the l_{NIR} value determined by the sensor height to get the grid elements around (X_{grid}, Y_{grid}) that need to have their NDVI values updated: $\Delta = \left\lfloor \frac{l_{NIR}}{1.6\ cm} \right\rfloor$ The NDVI for a grid element (X, Y) is updated if $\Delta \geq X - X_{grid}$ && $\Delta \geq Y - Y_{grid}$. For LiDAR readings, we maintain 16-bit counters for the number of plant hits and total hits in each grid element.

When the map receives an end signal from the GPS position interpolator indicating that the sensor has moved one metre from the starting point of the scan, we compute and output the global NDVI product sum of all grid elements, represented as a 48-bit integer. The calculation for the global NDVI product sum is:

$$NDVI_{global} = \sum_{i=0}^{4095} NDVI_i * (Plant\ hits)_i \tag{12}$$

Where i represents the grid element index. This calculation is implemented as a stream-out multiply-accumulate, which is executed after the end signal is received. The maximum NDVI product sum for a single cell requires 32 bits as it multiplies a 16-bit NDVI and a 16-bit hit count. We add 4096 = 2^{12} 32-bit values for each element in the grid, which gives us a 44-bit value in the worst case. We choose 48 bits to represent the global NDVI product sum since each DSP48E1 slice contains a 48-bit ALU and accumulator, and so that we can send the product to the MCU over exactly 6 bytes. We also output the global number of plant hits, which is used by the MCU to calculate the hit-weighted NDVI of the entire grid.

We considered using a density-weighted approach, where the global NDVI would be a sum of the NDVI values for each grid element times the number of plant hits divided by the number of total hits in each grid, but this would either involve a large amount of values being outputted to the MCU and requiring the MCU to spend 4096 cycles computing the global NDVI using data in each grid element, or the FPGA would have to compute a large and complex division operation to output the global NDVI.

3.2.3 Internal AXI Bus

We use an AXI bus as the method of communication between the sensor intake modules and the processing engines of the FPGA module. The necessity of an AXI bus is only apparent when we consider scalability for multiple RPLIDAR and AS7341 sensors. As the number of sensors increase, we need to arbitrate the data processing on sensor data as it may not be feasible to scale a processing engine to each set of sensors. The alternative is to manually implement an arbiter to handle sensor data packets, which may be costly in development time and error prone.

Vivado Design Suite provides a free-to-use AXI4-Stream IP block that can be easily instantiated into our design. This IP block has already been optimized and thoroughly verified, so it is a suitable solution to the scalability problem. The AXI protocol is also simple – the core functionality only requires each sensor to have data, valid, and ready wires to be integrated into the interconnect. We define the sensor ingestion module(s) as the AXI master(s) and the processing engine as the AXI slave of the system since the terminology is defined by the direction of data flow. The IP supports up to 16 masters and 16 slaves, which meets our demands for (**NFS9**), requiring support for multiple sensors.

The following table shows the mandatory signals for slave modules, which will be the processing engine of the FPGA subsystem [15].

Table XVII
Processing Engine AXI Bus Interface Signals

Signal Name	Direction	Description
ACLK	Input	Clock signal – all inputs and outputs align with the rising edge of ACLK
ARESETN	Input	Active-low synchronous reset signal
ACLKEN	Input	Clock enable signal
TVALID	Input	Indicates when a sensor module is driving valid data on the bus
TREADY	Output	Indicates that the processing engine is ready to receive data – when deasserted, backpressures data from the sensor integration module
TDATA	Input	Data payload
TLAST	Input	Indicates the end of a data packet – important for continuously streaming data

The mandatory signals for the master modules are the same as those for the slave modules, except the direction of TVALID, TREADY, TDATA, and TLAST are reversed.

3.2.4 MCU Interface

The FPGA subsystem requires two-way communication between the processing engine and the MCU. From the MCU to FPGA direction, we require configuration information as well as start and end signals for the window of data collection. At device initialization and while the device is not collecting data, the user may specify the operating height of the sensor. This value is used by the processing engine to determine the validity of the BRAM array element the point belongs in. The FPGA subsystem may also receive signals to start and end data collection, which determine the control signals the processing engine send to the sensor ingestion module.

All communication between the FPGA subsystem and the MCU are conducted over SPI. We use the Vivado AXI Quad SPI IP core to implement the slave side of SPI. This IP core is free-to-use in Vivado Design Suite and is more robust and optimized than a manual implementation of a SPI slave. The IP core can be configured as a SPI slave and customized as described in the Design Flow steps of the Xilinx product guide [14].

The following table describes the 16-bit packet format in the MCU to FPGA direction for setting sensor height and sending start/stop signals. Some bits have been reserved for flexibility in the future to implement other configuration settings.

Table XVIII
MCU to FPGA SPI Configuration Packet Format

Bits	Description	Value Definition
[15:14]	Command Header	00 = Set Sensor Height 01 = Set Plant Hit Threshold 10 = Start/Stop Data Collection
[13:6]	Reserved	N/A
[5:0]	Data	000001 = Start Data Collection when Header == 10 000000 = End Data Collection when Header == 10 Set Sensor Height to [Data * 5 cm] when Header == 00 Set Plant Hit Threshold to [Data * 5 cm] when Header == 01

The following table describes the seven 16-bit word format for the data packet the FPGA outputs to the MCU. This packet will be sent periodically to the MCU every 1 metre scanned.

Table XIX
FPGA to MCU SPI Telemetry Packet Structure

Word	Field Name	Data Representation	Description
Word 0	Start Frame	Constant uint16 (0xAAAA)	A constant identifier to notify MCU for the start of the packet
Word 1	Global NDVI Product Sum[47:32]	48-bit integer	Sum of the product of the NDVI times the number of plant hits for all grid cells
Word 2	Global NDVI Product Sum[31:16]		
Word 3	Global NDVI Product Sum[15:0]		
Word 4	Global Number of Plant Hits[31:16]	32-bit unsigned integer	Total number of LiDAR data points determined to be plant hits
Word 5	Global Number of Plant Hits[15:0]		
Word 6	Global Number of Hits[31:16]	32-bit unsigned integer	Total number of LiDAR data points
Word 7	Global Number of Hits[15:0]		

3.3 Microcontroller

The TerraScan core processing is driven by an STM32G474RE microcontroller, featuring an ARM Cortex M4 core running at a peak clock frequency of 170MHz. The subsystems primary constraints are governed by a strict 128 KB internal SRAM ceiling and a real time requirement to take in high bandwidth, asynchronous metric bursts from the FPGA without dropping packets or inducing memory corruption.

3.3.1 Architectural Trade-offs and Constraints

A traditional single-threaded polling architecture was analyzed and rejected for this design. Local data serialization via the CH376 storage module and wireless telemetry transmissions across the AT-09 Bluetooth interface introduce blocking hardware I/O delays on the order of 10's of milliseconds. If a single-threaded loop is engaged in a data store or wireless transmission when the FPGA completes a scan, the microcontroller will fail to sample the physical data, leading to data drops during high-density operation.

To eliminate this bottleneck, the software architecture utilizes a preemptive FreeRTOS kernel. The high-level system state machine responsible for ingesting and processing FPGA data bursts is isolated from slow asynchronous peripheral I/O through a strictly event-driven task structure. A dedicated "hardware driver server" execution thread was also evaluated and rejected, as it introduces real-time context-switching overhead that conflicts with the SRAM constraint. Instead, all peripheral drivers are implemented as re-entrant functions protected by native RTOS binary semaphores and lightweight memory pointers, preserving a deterministic execution profile.

3.3.2 Task Allocation & Priorities

The software system splits its operations into three independent, preemptive tasks:

1. **VMainStateTask (Priority: High, configMAX_PRIORITIES – 1)**: Handles the core state machine and FPGA data ingestion. The task remains blocked using direct-to-task notification, unblocking instantly when an FPGA hardware interrupt asserts the notification. Direct-to-task notification was chosen over a binary semaphore as it is roughly 45% faster and uses significantly less RAM.
2. **VStorageTask (Priority: Medium, 1)**: Isolates local storage media latency. It remains blocked on an asynchronous queue (xStorageQueue) and wakes to send data packets to the CH376 module for storage.
3. **VBluetoothTask (Priority: Low, 0)**: Handles diagnostics and control message broadcasting via the AT-09 module. It remains blocked on an asynchronous queue (xBluetoothQueue) and wakes to transmit data packets wirelessly.

3.3.3 Inter-Task Communication Mechanics

Tasks communicate exclusively via thread-safe, FIFO FreeRTOS queues (xStorageQueue and xBluetoothQueue). To avoid memory copy overhead within the SRAM constraint, the high-priority state machine never duplicates data structures. It packages sensor metrics into a unified ScanPacket_t structure and pushes only the memory pointer into the downstream queues, keeping context-switching overhead and stack size small.

To avoid race conditions without the overhead of mutexes, a dedicated circular buffer stores available memory addresses for packets. When the FPGA signals data ready, vMainStateTask pops

the next available address from this buffer and populates it. Once both vStorageTask and vBluetoothTask finish processing the pointer, the final task returns the address to the free pool.

If a downstream queue experiences delays from a slow peripheral write and the circular buffer runs out of available blocks, the system safely drops the current packet and returns a packet loss warning.

3.3.4 State Machine Architecture & Transitions

The VMainStateTask operates as a deterministic Finite State Machine (FSM) divided into four operational states, each detailed below with its transition mechanics.

3.3.4.1 Idle State (ST_IDLE)

Entered immediately following power-on and restarts. The device performs minimal processing, no FPGA data is received, and only environmental and GPS data is collected and sent over Bluetooth while the system waits for a scan-start command

Table XX
Idle State (ST_IDLE) Transition Chart

Current State	Event / Condition	Transition Actions	Next State
ST_IDLE	EVT_SCAN_START	(1) Send command received ACK over Bluetooth. (2) Write file start identifier to storage media. (3) Send start scan command to FPGA.	ST_RUNNING
ST_IDLE	COND_FAULT	(1) Write fault code to storage media. (2) Send fault code over Bluetooth.	ST_ERROR

3.3.4.2 Running State (ST_RUNNING)

The core operational state, where the system ingests, processes, and packages real-time sensor and FPGA metrics for storage and transmission.

Table XXI
Running State (ST_RUNNING) Transition Chart

Current State	Event / Condition	Transition Actions	Next State
ST_RUNNING	EVT_FPGA_DATA_READY	(1) Retrieve free memory block from free buffer. (2) Insert FPGA data into packet. (3) Retrieve GPS data and insert into packet. (4) Retrieve Temperature & Humidity and insert into packet. (5) Dispatch memory pointer to xStorageQueue and xBluetoothQueue.	ST_RUNNING
ST_RUNNING	EVT_APP_SHUTDOWN	(1) Send command received ACK over Bluetooth. (2) Send stop scan command to FPGA.	ST_SHUTDOWN
ST_RUNNING	COND_FAULT	(1) Send stop scan command to FPGA. (2) Write fault code to storage media. (3) Send fault code over Bluetooth.	ST_ERROR

3.3.4.3 Shutdown State (ST_SHUTDOWN)

Executes clean-up procedures before returning to ST_IDLE, preventing new scans while ensuring remaining data packets are fully processed, stored, and transmitted.

Table XXII
Shutdown State (ST_SHUTDOWN) Transition Chart

Current State	Event / Condition	Transition Actions	Next State
ST_SHUTDOWN	COND_QUEUES_NOT_EMPTY	(1) Continue processing of remaining packets.	ST_SHUTDOWN
ST_SHUTDOWN	COND_QUEUES_EMPTY	(1) Write file end identifier to storage media. (2) Send shutdown complete command over Bluetooth.	ST_IDLE
ST_SHUTDOWN	COND_FAULT	(1) Write fault code to storage media. (2) Send fault code over Bluetooth.	ST_ERROR

3.3.4.4 Error State (ST_ERROR)

A dedicated safety trap entered on any fault condition. All scanning operations are disabled to protect data integrity, and the system broadcasts diagnostic alerts until a manual clear command is received from the dashboard.

Table XXIII
Error State (ST_ERROR Transition Chart

Current State	Event / Condition	Transition Actions	Next State
ST_ERROR	EVT_CLEAR_FAULT	(1) Check if fault condition is still present. (2) Clear fault identifier.	ST_IDLE
ST_ERROR	COND_FAULT	(1) Write fault code to storage media. (2) Send fault code over Bluetooth.	ST_ERROR

Note: Any event or condition not explicitly listed in a state's transition table has no operational effect while the system is in that state and is treated as a No Operation (NOP).

3.4 Main PCB

Figure 2 shows the electrical schematic for the initial prototype of our TerraScan device.

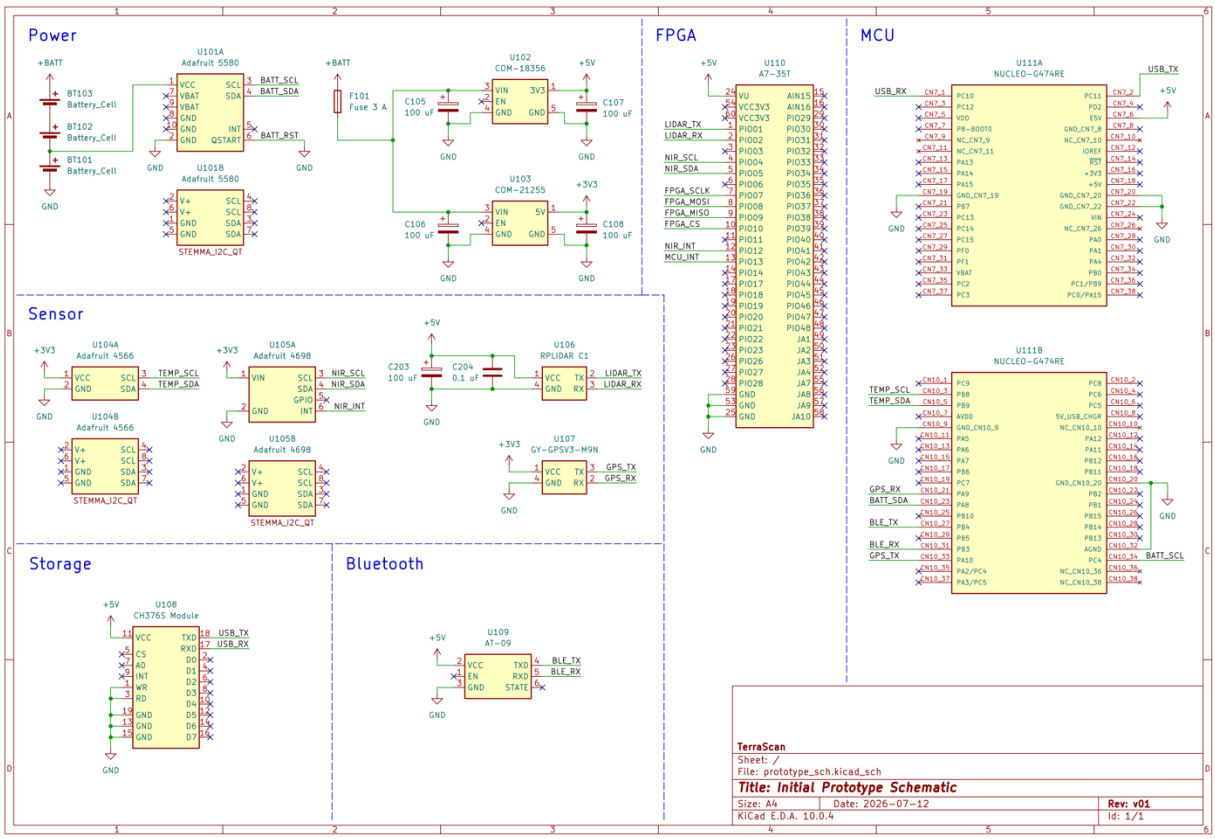


Fig. 2. TerraScan Initial Prototype Schematic.

3.4.1 Power Delivery

Since the device is mounted to a mobile platform, a battery is required to provide power. To meet the 8-hour operating requirement (**NFS1**), the device will be powered by a 5200 mAh, 11.1 V, 3S LiPo rechargeable battery, fused at 3 A and stepped down via separate buck converters to 5 V and 3.3 V rails. The 5 V rail powers the LiDAR sensor, USB module, FPGA, Bluetooth module, and microcontroller; the 3.3 V rail powers the NIR sensor, GNSS module, and temperature & humidity sensor. Decoupling capacitors will be used on both rails to prevent buck-converter switching noise from reaching downstream devices, with extra decoupling near the LiDAR sensor to isolate motor noise from the FPGA. Additional decoupling upstream of the converters will prevent current spikes from tripping the fuse or causing battery voltage drops.

3.4.1.1 Part Decisions

Battery: An 8000 mAh, 7.4 V 2S LiPo was the original choice, offering ~8.5 hours of runtime at our estimated 7 W draw. It was dropped because a depleted 7.4 V pack drops to ~6.4 V, leaving too little margin for efficient 5 V step-down, and larger 2S batteries are hard to source affordably. The 5200 mAh, 11.1 V 3S LiPo was chosen instead: similar runtime (~8.25 hrs), a healthier depleted voltage margin (~9.6 V) for the buck converter, and a lower-capacity, cheaper battery that leaves room for future capacity upgrades.

Voltage Step-down: The TI TPS563203 was the initial choice for both 5 V and 3.3 V conversion, but it isn't available on affordable breakout boards for prototyping. We moved to the Diodes Inc. AP63203 and AP63357, which offer a wider input voltage range and comparable efficiency/current limits, and are available on SparkFun breakout boards suitable for our prototype.

Battery Monitoring: The Maxim MAX17205 fuel gauge was the original choice for combined coulomb-counting and voltage-based SoC tracking, but no affordable breakout board was available. We instead use the simpler MAX17048, which is designed for 1S 3.7 V packs and can only measure one cell of our 3S battery. To compensate, we pair it with a voltage-divider circuit feeding an MCU ADC channel to monitor overall pack voltage. The MCU reads the MAX17048 over I2C (PC4 – SCL, PA8 – SDA) and uses the ADC reading to trigger shutdown on critical undervoltage.

3.4.2 MCU Peripherals

The MCU Peripherals block covers the four external peripherals wired into the microcontroller (STM32G474RE) over UART and I2C (AT-09 Bluetooth module, CH376 USB storage controller, Adafruit 4566 Temp/Humidity sensor, NEO-M9N-00B GPS receiver). Each part below was chosen over at least one alternative that was seriously considered and ruled out.

3.4.2.1 Part Decisions

Bluetooth Module: The HC-05 Bluetooth Classic SPP module was originally considered for its low cost and documentation, but was dropped because iOS blocks third-party SPP support, which would have limited the companion app to Android. The AT-09 is a BLE (Bluetooth Low Energy) device, compatible with both iOS and Android with no licensing requirement.

USB Storage:
Writing to an SD card directly over the MCU's SPI bus (using FatFs) was the original approach, but was rejected because it requires maintaining a filesystem implementation on the MCU itself, adding unwanted firmware complexity. The CH376 instead handles FAT32/FAT16/FAT12 locally and only needs simple serial commands from the MCU. Its USB form factor is also more accessible to the average consumer, better satisfying (**NFS5**).

Temperature/Humidity sensor: The DHT11 was the original choice for its low cost, but its accuracy (±2 °C / ±5% RH) doesn't meet (**FS8**) and was eliminated on that basis. The Adafruit AHT20 was selected instead, rated at ±0.3 °C / ±2% RH, satisfying (**FS8**). It also uses I2C rather than the DHT family's single-wire bit-banging protocol, avoiding the need for a custom low-level driver.

GPS Module: Low-cost patch antenna modules like the NEO-6M were considered but rejected, they support only single-constellation GPS and a slow 5 Hz refresh rate, insufficient for maintaining positioning accuracy during fast-paced use. The u-blox NEO-M9N was selected instead: it tracks up to four constellations (GPS, GLONASS, Galileo, BeiDou) at up to 25 Hz refresh, with 2.0 m horizontal accuracy, satisfying (**FS7**).

3.4.2.2 Peripheral Pin Mapping

Table XXIV lists the hardware peripheral pin assignments and communication protocol mappings for the MCU Peripherals subsystem.

Table XXIV
MCU Peripheral Hardware Pin and Protocol Mapping

Device	Protocol	MCU Peripheral	Pin(s)
AT-09 BLE	UART (async.)	USART2	PB3 (TX), PB4 (RX)
CH376 USB Module	UART (async.)	USART3	PC10 (TX), PC11 (RX)
AHT20 Temp/Humidity Sensor	I2C (master)	I2C1	PB8 (SCL), PB9 (SDA)
GPS Receiver (NEO-M9N)	UART (async.)	USART1	PA9 (TX), PA10 (RX)

3.4.2.3 Interface Level Design

USART2 – AT-09 BLE Module: Configured at 9600 baud (factory default), 8N1 framing, per Table xxx. This link is the physical transport for the companion app dashboard (**FS10**) and handles periodic system telemetry and input commands (**FS9**).

USART3 – CH376 USB Module: Initializes at 9600 baud (8N1) to establish the initial serial handshake, after which the MCU commands a transition to 57,600 baud to increase sustained logging throughput. The CH376 translates simple serial commands into local FAT32 filesystem operations, supporting loss-free non-volatile writes (**FS6**) and (**NFS5**).

I2C1 – AHT20 Temperature/Humidity Sensor: Operates in Fast Mode at 400 kHz with a 7-bit address of 0x38. Fast Mode is used to minimize bus transaction time and reduce MCU polling overhead. The sensor's baseline accuracy (±0.3 °C / ±2% RH) satisfies (**FS8**). Because the MCU's I2C pins are open-drain and internal pull-ups are too weak at 400 kHz, two external 2.2 kΩ pull-up resistors are added to SCL and SDA.

USART1 – NEO-M9N GPS Interface: Configured at 9600 baud, 8N1 framing, to continuously ingest NMEA-0183 sentences providing real-time coordinates and timestamp metadata. The module operates at native 3.3 V I/O, matching the STM32's CMOS pins, and includes an internal backup battery to retain satellite almanac data across power cycles for fast hot-start tracking.

3.5 Software System

3.5.1 Companion Dashboard Architecture

To satisfy the companion application requirement (**FS10**) without adding unnecessary development risk, we decided to build a desktop dashboard that runs on a laptop rather than writing a full mobile app from scratch. The laptop connects to the AT-09 wirelessly over Bluetooth, which the laptop's OS maps as a virtual serial COM port so that our software can easily read the data stream.

Since the hardware link just passes standard serial data over a basic Bluetooth BLE profile, the firmware side is completely separated from the UI side, allowing the interface to be replaced or extended without modifying the STM32 code or hardware.

3.5.2 Dashboard User Interface Layout & Commands

The desktop interface is built using Python (with customtkinter for a clean UI layout and pyserial to catch the incoming data stream). We split the layout into three basic functional sections.

1. **Session Controls & State Indicators**:
   - "Start Scan" and "Stop Scan" Buttons: These buttons let the user control the system by sending simple ASCII text commands ("START\n" and "STOP\n") over the Bluetooth connection to toggle the STM32's internal state machine.
   - Device Configuration Controls: The dashboard will have a settings panel where users can input specific operational parameters before starting a scan, like setting the scanning height (in meters). When the user updates this value, the dashboard sends a configuration string to the MCU so it can update its active configurations.
   - Scanning Status Indicator: The dashboard will have a clear visual status badge to show if the system is actively scanning or idle. To prevent false readouts, this indicator does not toggle when the user clicks the scan buttons. It stays in a pending state until the

laptop receives an explicit confirmation message from the MCU confirming the device has safely switched states.

2. **Real Time GPS Stream**: The dashboard will display live readouts of the current UTC timestamp and decoded coordinates (latitude and longitude) while the device is on.
3. **Environmental Telemetry**: The dashboard will display real time temperature (°C) and relative humidity (%RH) when the device is on.
4. **Fault Status & Diagnostics**: A dedicated error monitoring section tracks system health. If the device encounters an error, it sends the error code over Bluetooth, and the dashboard decodes and displays it with a clear, human-readable description.

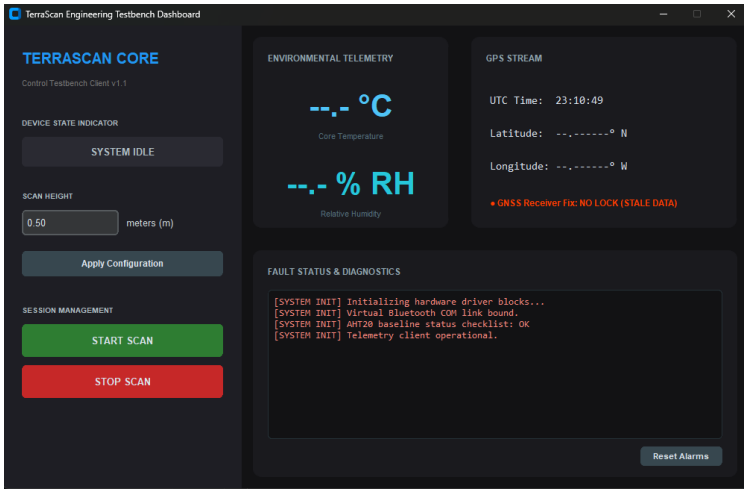


Fig. 3. TerraScan Testbench Dashboard UI.

3.5.3 Automated CSV Data Logging Pipeline

To provide a reliable backup alongside the onboard USB storage, the laptop application handles its own automated background data logging.

Every time a scan starts and the application receives confirmation, the Python script generates a new .csv file in a dedicated local folder (/scans/). Each file is titled with a standard timestamp format (scan_YYYYMMDD_HHMMSS.csv) to avoid overwriting older test runs.

As data frames arrive over the Bluetooth port, a background thread validates the data, strips the serial framing tokens, adds a local timestamp for when the laptop received it, and writes the entire data packet directly into the open CSV file. When the user stops the scan and the confirmation message is received, the app flushes the remaining data buffers and safely closes the file.

3.5.4 Implementation Scope and Future Roadmap

Native mobile apps for iOS and Android, or a commercial-grade desktop software package, are the obvious next steps for the product but are outside the scope of this project. Because the physical design relies entirely on a universal BLE setup broadcasting standard serial lines, any future app development can be implemented as a software-only update, requiring no changes to the STM32 firmware or hardware.

4 Discussion and Project Timeline

4.1 Evaluation of Final Design

The proposed final design successfully satisfies most of the project objectives and specifications. Every major hardware choice maps directly to a primary functional or non-functional requirement.

The functional specifications for dual-band spectral sensing (**FS2**, **FS4**) and canopy density tracking (**FS1**, **FS3**) are met via the AS7341 spectral sensor and the RPLIDAR C1. The NIR channel center wavelength is at 910 nm; while slightly outside the ideal 800–900 nm window specified in **FS2**, it remains highly sensitive to canopy reflectance. Real time data ingestion and processing (**FS5**, **FS6**) are handled by routing metrics through thread safe FreeRTOS queues on the STM32G474RE, eliminating blocking I/O delays within the 128 KB SRAM limit. Geographical and environmental contexts (**FS7**, **FS8**) are satisfied via the u-blox NEO-M9N GNSS module and the AHT20 sensor. To manage timeline risks, the user interface (**FS10**) was pivoted from a mobile application to a Python-based desktop testbench dashboard, which still complies with the core requirements of displaying system states and logging data.

For the non-functional and interface specifications, data offloading satisfies the physical connection requirements via a USB interface (**NFS5**). Power consumption (**NFS6**) and battery life (**NFS1**) are addressed by the 3S LiPo battery pack design. Due to breakout board sourcing constraints for the prototype, a 1S MAX17048 fuel gauge is paired with a hardware voltage divider and an ADC channel to monitor and protect the 3S pack.

In terms of physical implementation, the design does not yet incorporate specific enclosure or mounting hardware considerations to meet durability (**NFS3**) and IP65 ingress protection (**NFS8**) standards. Because the immediate focus is establishing electrical and firmware functionality, satisfying these environmental specs is not a concern for the current prototyping phase. High level structural design, environmental sealing gaskets, and vibration-isolation mounts are deferred to the final design and production phase.

4.2 Use of Advanced Knowledge

This project draws on upper-year ECE knowledge across digital hardware, embedded systems, and circuit design. The FPGA-based processing pipeline requires knowledge from ECE 327 (Digital Hardware Systems) for designing and verifying the HDL logic that interfaces with and synchronizes data from the LiDAR and NIR sensors. This involves designing digital logic to correctly capture, process, and pass along data from both sensors so that readings can be reliably combined and used downstream. Processing of the sensor data applies concepts from ECE 313 (Digital Signal Processing) in implementing an IIR filter to compute an exponential moving average of the sensor readings. The microcontroller firmware, which handles real-time data acquisition and communication between the microcontroller, FPGA, and sensors, draws on ECE 455 (Embedded Software), applying embedded software design principles to coordinate data transfer between components while meeting real-time timing constraints. Designing the custom PCB requires knowledge of analog and digital circuit design from ECE 340 (Electronic Circuits), including component selection and circuit analysis for interfacing the sensor front-ends with the digital FPGA/microcontroller sections. This ensures the various components can communicate correctly and operate reliably as an integrated system. To ensure high-speed digital signals route without degradation, the PCB design also applies transmission line theory from ECE 375 (Electromagnetic Fields and Waves) to manage trace impedance and mitigate signal reflections.

4.3 Creativity, Novelty, Elegance

The most elegant aspect of this design is the hardware-software co-design, utilizing a custom PCB that cohesively pairs an FPGA with a microcontroller. This architecture perfectly balances high-throughput processing with efficient system control. Instead of bottlenecking a standard processor, the FPGA acts as a hardware accelerator. It intakes dense, real-time spatial data from the LiDAR alongside spectral data from the NIR sensor, computing the NDVI algorithm in parallel.
The primary novelty lies in transforming static agricultural scanning into a dynamic, mobile solution directly mounted to a tractor. By combining spatial depth mapping with vegetative health indices on the move, the system correlates precise 2D distance coordinates directly with computed NDVI values to yield highly localized crop diagnostics. Ultimately, this embedded approach delivers a robust precision agriculture tool that utilizes heavy sensor fusion locally, providing a highly cost-effective alternative to expensive commercial systems while eliminating the need for continuous network connectivity.

4.4 Student Hours

Omitted from the public version of this report.

4.5 Potential Safety Hazards

In compliance with the Criminal Code of Canada (Section 219), PEO professional misconduct guidelines (Reg. 941, Section 72), and University of Waterloo Policy 34, the system's design must actively minimize risks to life, health, property, and public well-being. The specific hazards identified for this device are dictated by its low voltage hardware and the unstructured agricultural environment in which it operates.

Battery Short Circuit and Thermal Hazards: Since the device operates on a low voltage DC rail there is no significant human electrical shock hazard. However, an accidental low resistance short circuit caused by field vibrations or wire insulation wear can cause massive current dumps. This massive current can cause damage to PCB traces or cause localized electrical/chemical fires of the device or the battery. Mitigation: The risk will be controlled by hardwiring fast acting inline fuses rated for 1.25 times the maximum continuous load current in order to disconnect the battery in the event of one of these faults.

Optical Hazards: Operating active scanning sensors exposes users and bystanders to optical radiation. Direct exposure to the eye can cause permanent retinal damage or localized blindness. Mitigation: Because our optical sensors remain idle at power up, this risk is handled via the system state machine. The Devices are only enabled upon an explicit scan start serial command from the FPGA, and any detected system exception triggers a fault transition that immediately transmits a command to disable the devices.

Environmental and Chemical Contamination Hazards: Operating in open field conditions exposes the device to heavy dust, high thermal loads and hazardous agricultural chemical residues. Ingress can cause trace level short circuits, while external surface residues present a skin contact hazard to operators. Mitigation: To completely block contaminant ingress, the main electronics bay is sealed inside an impact resistant enclosure engineered with a tight mechanical rubber gasket to satisfy (**NFS8**) (IP65 Ingress Protection) against fine particulates and fluid drift.

4.6 Project Timeline

Figure 4 shows the proposed project timelines for ECE498A (Spring 2026) and ECE498B (Winter 2027).

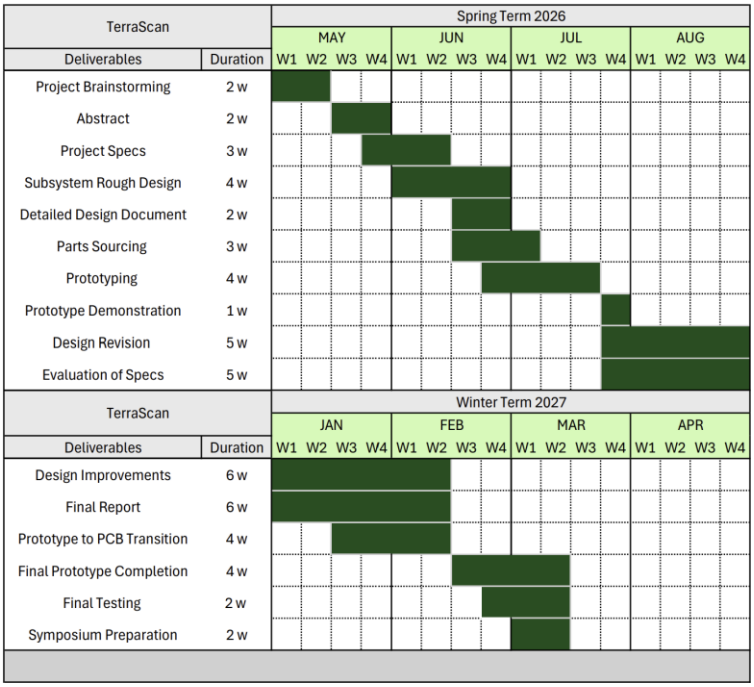


Fig. 4. Project Timeline for ECE498A and ECE498B.

References

[1] B. T. D. Institute, “Sustainable Agriculture Technology Adoption - barriers and enablers: The role of Skills,” Toronto Metropolitan University (TMU), https://www.torontomu.ca/diversity/news-events/2025/12/sustainable-agriculture-technology-adoption/ (accessed Jun. 14, 2026).

[2] U. S. G. A. Office, “U.S. GAO,” Precision Agriculture: Benefits and Challenges for Technology Adoption and Use, https://www.gao.gov/products/gao-24-105962 (accessed Jun. 10, 2026).

[3] S. M. Farhan, J. Yin, Z. Chen, and M. S. Memon, “A comprehensive review of lidar applications in crop management for Precision Agriculture,” MDPI, https://www.mdpi.com/1424-8220/24/16/5409 (accessed Jun. 10, 2026).

[4] R. P. Sishodia, R. L. Ray, and S. K. Singh, “Applications of remote sensing in Precision Agriculture: A Review,” MDPI, https://www.mdpi.com/2072-4292/12/19/3136 (accessed Jun. 10, 2026).

[5] G. Rabatel, N. Gorretta, and S. Labbé, “Getting ndvi spectral bands from a single standard RGB digital camera: A methodological approach,” Lecture Notes in Computer Science, pp. 333–342, 2011. doi:10.1007/978-3-642-25274-7_34

[6] B. Basener, "Neural Network Learning Of Chemical Bond Representations In Spectral Indices And Features," 2022 12th Workshop on Hyperspectral Imaging and Signal Processing: Evolution in Remote Sensing (WHISPERS), Rome, Italy, 2022, pp. 1-9, doi: 10.1109/WHISPERS56178.2022.9955112.

[7] J. Uribe-Rios, D. Marquez-Viloria, L. Castano-Londono, and M. C. Torres-Madronero, “FPGA-based four-band Multispectral Camera Prototype for precision agriculture applications,” Communications in Computer and Information Science, pp. 255–269, 2023. doi:10.1007/978-3-031-32213-6_19

[8] “NDVI, the foundation for Remote Sensing Phenology,” USGS, https://www.usgs.gov/special-topics/remote-sensing-phenology/science/ndvi-foundation-remote-sensing-phenology (accessed Jun. 6, 2026).

[9] AMD, "7 Series FPGAs Data Sheet: Overview (DS180)," AMD, https://docs.amd.com/v/u/en-US/ds180_7Series_Overview (accessed Jun. 14, 2026).

[10] AMD, "Artix 7 FPGAs Data Sheet: DC and AC Switching Characteristics (DS181)," AMD, https://docs.amd.com/v/u/en-US/ds181_Artix_7_Data_Sheet (accessed Jun. 14, 2026).

[11] Shanghai Slamtec Co., Ltd., "RPLIDAR C1: Low-Cost 360 Degree Laser Range Scanner — Introduction and Datasheet," Shanghai Slamtec Co., Ltd., 2023. [Online]. Available: https://d229kd5ey79jzj.cloudfront.net/3157/SLAMTEC_rplidar_datasheet_C1_v1.0_en.pdf (accessed Jun. 14, 2026).

[12] ams AG, "AS7341 11-Channel Multi-Spectral Digital Sensor Datasheet (DS000504)," ams AG, 2020. [Online]. Available: https://cdn.sparkfun.com/assets/0/8/e/2/3/AS7341_DS000504_3-00.pdf (accessed Jun. 11, 2026).

[13] AMD, "LogiCORE IP CORDIC v5.0 Data Sheet (AXI) (DS858)," AMD, https://docs.amd.com/v/u/en-US/ds858_cordic (accessed Jun. 12, 2026).

[14] AMD, "AXI Quad SPI v3.2 LogiCORE IP Product Guide," AMD, https://docs.amd.com/viewer/book-attachment/X_lSgvYp93ukDgBX9nUxUA/nQejvD0tvMiQ1XcbzLKH4g-X_lSgvYp93ukDgBX9nUxUA (accessed Jun. 16, 2026).

[15] AMD, "AXI4-Stream Interconnect v1.1 Product Guide (PG035)," AMD, https://docs.amd.com/v/u/en-US/pg035_axis_interconnect (accessed Jun. 14, 2026).

[16] Shanghai Slamtec Co., Ltd., "RPLIDAR: 360 Degree Laser Range Scanner Interface Protocol and Application Notes — Applied to RPLIDAR S Series," Shanghai Slamtec Co., Ltd., 2023. [Online]. Available: https://core-electronics.com.au/attachments/localcontent/series_protocol_LR001_SLAMTEC_rplidar_series_protocol_v1_22332493c35.pdf (accessed Jun. 25, 2026).

[17] B. Siepert, "Adafruit_AS7341.cpp," Adafruit_AS7341 GitHub repository, Adafruit Industries, 2020. [Online]. Available: https://github.com/adafruit/Adafruit_AS7341/blob/master/Adafruit_AS7341.cpp (accessed Jun. 21, 2026).

[18] AMD, "7 Series DSP48E1 Slice User Guide (UG479)," AMD, https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1 (accessed Jun. 21, 2026).

[19] P. Padmanabhan, C. Zhang, and E. Charbon, “Modeling and Analysis of a Direct Time-of-Flight Sensor Architecture for LiDAR Applications,” *Sensors*, vol. 19, no. 24, p. 5464, Dec. 2019, doi: 10.3390/s19245464.

[20] B. Siepert and K. Rembor, "Adafruit AS7341 10-Channel Light / Color Sensor Breakout," Adafruit Learning System, https://learn.adafruit.com/adafruit-as7341-10-channel-light-color-sensor-breakout/overview (accessed Jul. 2, 2026).

[21] NXP Semiconductors, "UM10204: I^2C-Bus Specification and User Manual," Rev. 7.0, NXP Semiconductors, 2021. [Online]. Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf (accessed Jul. 2, 2026).